

Reflections for WordGuess program

How did you approach understanding the challenge?

I approached the challenge by breaking the program into smaller parts, such as file reading, random word selection, and user interaction. By analyzing each section of the code, I developed a clearer understanding of how the program flows. For example, the file-reading section selects a random word using **Math.random()**, while later parts handle user input and update the displayed word.

```
// Generate random line number between 1 and 20
int randomNum = (int) (20 * (Math.random()) + 1);
```

How did you decide on the tools, data structures, or algorithms to use?

I chose to use Java file-handling classes such as **FileReader** and **BufferedReader** to read words from a file. I used **Math.random()** to randomly select a word, as seen in the code where a random line number is generated. Additionally, I used arrays (**letterCounts**) to track guessed letters and prevent repeated guesses.

```
// Array to track how many times each letter is guessed
int[] letterCounts = new int[HIGH - LOW + 1];
```

```
FileReader in;           // Reads characters from file
BufferedReader readFile; // Reads file line-by-line
```

How did you test your solution along the way to make sure it was working?

I tested the program frequently by running it after implementing each major section. For example, I tested whether the correct word was selected from the file and whether the displayed word updated properly after each guess. I also checked edge cases, such as entering multiple letters or guessing the same letter twice.

```
// Ask player for input
System.out.println("Enter a letter (" + FLAG + " to guess entire word): ");
```

```
// Show current progress
System.out.println(wordSoFar + "\n");
```

```
// ----- LETTER ALREADY GUESSED -----
else if (letterCounts[countletters] > 1){
    System.out.println("You already guessed this letter.\n");
```

What part of the problem was the most difficult for you?

The most difficult part was managing user input and updating the word correctly. Specifically, ensuring that the guessed letter appears in the correct positions (as shown in the loop updating **wordSoFar**) required careful logic. Handling repeated guesses using the **letterCounts** array was also challenging.

```
// ----- CORRECT LETTER GUESS -----
if (SECRET_WORD.indexOf(letterGuess) >= 0 && letterCounts[countletters] <= 1) {

    updatedWord = ""; // Reset updated word

    // Loop through each letter in secret word
    for (int i = 0; i < SECRET_WORD.length(); i++) {

        // If guessed letter matches position in word
        if (SECRET_WORD.charAt(i) == letterGuess.charAt(0)) {
            updatedWord += letterGuess; // Reveal letter
        }
        else {
            updatedWord += wordSoFar.charAt(i); // Keep previous state
        }
    }

    // Update displayed word
    wordSoFar = updatedWord;
}
```

Was there anything you learned that you think will help you with future challenges?

I learned how to combine file handling, randomness, and user interaction in one program. This project improved my understanding of arrays, loops, and exceptions, which will help in building more complex interactive programs.

```
// Error if file not found
catch(FileNotFoundException e)
{
    System.out.println("File does not exist or could not be found");
    System.err.println("FileNotFoundException: " + e.getMessage());
}

// Error while reading file
catch(IOException e)
{
    System.out.println("Problem reading file.");
    System.err.println("IOException: " + e.getMessage());
}
```

Reflections for CountVowels program

How did you approach understanding the challenge?

I approached the challenge by focusing on how to process text from a file step-by-step. I first understood how to read a file line by line, then how to analyze each character in the string. Breaking the problem into reading, processing, and counting made it easier to understand.

```
// Read file line by line until there are no more lines
while ((lineOfText = readFile.readLine()) != null)
```

How did you decide on the tools, data structures, or algorithms to use?

I used **FileReader** and **BufferedReader** to read the file efficiently. To count vowels, I used a loop that iterates through each character using **substring(i - 1, i)**. I also used **toLowerCase()** to ensure the program counts vowels regardless of capitalization. No complex data structures were needed, but string manipulation was essential.

```
// Convert text to lowercase so vowel check is case-insensitive
lowercaseText = text.toLowerCase();
```

```
// Loop through each character in the text
letter = lowercaseText.substring(i - 1, i);
```

```
FileReader in;
BufferedReader readFile;
```

How did you test your solution along the way to make sure it was working?

I tested the program using different text files to ensure accuracy. I verified that uppercase and lowercase vowels were counted correctly and checked the final output.

```
// Output total number of vowels found in the file
System.out.println("\nThe number of vowels in the file text is " + vowelSum);
```

What part of the problem was the most difficult for you?

The most difficult part was correctly looping through each character in the string and avoiding indexing errors. Understanding why `i - 1` is used in `substring()` (because Java indexing starts at 0) was important.

```
// Loop through each character in the line
for (int i = 1; i <= lowercaseText.length(); i++)
{
    // Extract one character at a time using substring
    // (i-1 because Java uses 0-based indexing)
    letter = lowercaseText.substring(i - 1, i);

    // Check if the character is a vowel
    if (letter.equals("a") || letter.equals("e") ||
        letter.equals("o") || letter.equals("i") ||
        letter.equals("u"))
    {
        // If it is a vowel, increase the count
        vowelSum += 1;
    }
}
```

Was there anything you learned that you think will help you with future challenges?

I learned how to work with file input and string processing in Java. This will help me in future tasks that involve analyzing or manipulating text data.

```
// Ask user to enter file path
System.out.println("Enter the path to a file(for example: C:\\Users\\User\\Documents\\Text.txt) : ");
String filename = input.nextLine().trim(); // Remove extra spaces

// Create File object using the path
File textFile = new File(filename);
```

Reflections for StudentSemesterAverageGUI program

How did you approach understanding the challenge?

I approached this challenge by focusing on how GUI components interact with user input. I analyzed how text fields collect data and how action listeners trigger specific operations like saving or displaying data.

```
 JButton saveButton = new JButton("Save to File");
 saveButton.setFont(new Font("SimSun", Font.PLAIN, 16));
 saveButton.setBounds(124, 392, 147, 49);
 frame.getContentPane().add(saveButton);
 saveButton.addActionListener(new ActionListener() {
     public void actionPerformed(ActionEvent e)
     {
```

```
 // Main window
 private JFrame frame;

 // Input fields
 private JTextField sname;
 private JTextField gradelvl;
 private JTextField semestr;
 private JTextField grade1;
 private JTextField grade2;
 private JTextField grade3;
 private JTextField grade4;
```

How did you decide on the tools, data structures, or algorithms to use?

I used Java Swing components such as **JFrame**, **JTextField**, **JButton**, and **JTextArea** to build the interface. I applied methods like **.trim()** to clean user input and **Double.parseDouble()** to convert input into numeric values. Conditional statements were used to check inputs (e.g., grade ranges and empty fields).

```
 // Get user input
 String sNameInput = sname.getText().trim();
```

```
 double grdlvl = Double.parseDouble(gradelvlInput);
```

```
 JButton btnViewFileContents = new JButton("View File Contents");
```

```
 // Text area to display file content or messages
 JTextArea fileInfo = new JTextArea();
```

```
 // Input fields
 private JTextField sname;
```

How did you test your solution along the way to make sure it was working?

I tested the program by entering different types of input, including valid data, empty fields, and invalid numbers. I ensured that error messages appeared correctly and that valid data was saved to the file.

```
JOptionPane.showMessageDialog(null, "Data Saved Successfully!");  
  
// Validate empty fields  
if(sNameInput.isEmpty() || grdelvlInput.isEmpty() || semestrInput.isEmpty() ||  
    grade1Input.isEmpty() || grade2Input.isEmpty() || grade3Input.isEmpty() || grade4Input.isEmpty())  
{  
    fileInfo.append("Please fill in all fields.");  
    return;  
}
```

What part of the problem was the most difficult for you?

The most difficult part was handling user input and ensuring proper validation before saving data. Implementing action listeners that correctly respond to button clicks and execute multiple steps (calculation, file writing) was also challenging.

```
// Validate ranges  
if(grdlvl > 12 || grdlvl < 1)  
{  
    fileInfo.append("Grade level must be 1-12");  
}  
else if(semestr > 2 || semestr < 1)  
{  
    fileInfo.append("Semester must be 1-2");  
}  
else if(grd1 > 100 || grd1 < 0 || grd2 > 100 || grd2 < 0 || grd3 > 100 || grd3 < 0 || grd4 > 100 || grd4 < 0)  
{  
    fileInfo.append("Grades must be 0-100");  
}
```

```
// Validate empty fields  
if(sNameInput.isEmpty() || grdelvlInput.isEmpty() || semestrInput.isEmpty() ||  
    grade1Input.isEmpty() || grade2Input.isEmpty() || grade3Input.isEmpty() || grade4Input.isEmpty())  
{
```

Was there anything you learned that you think will help you with future challenges?

I learned how to build a functional GUI application that interacts with files. I also improved my understanding of event-driven programming, which are important for real-world applications.

```
// Write formatted data  
writeFile.write("Name: " + sNameInput + ", Grade Level: " + grdelvlInput +  
    ", Semester: " + semestrInput + ", Grades: " + grade1Input + ", " +  
    grade2Input + ", " + grade3Input + ", " + grade4Input + ", " + "Average: " + score);  
writeFile.newLine();
```